

Premiga pengtab 673040672-0

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
from google.colab import files
uploaded = files.upload()
```

 Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving diabetes.csv to diabetes.csv

```
df = pd.read_csv('diabetes.csv')
X = df.drop('Outcome', axis=1)
y = df['Outcome']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
# Scale the features, which is important for neural networks
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
#Experiment with Structure 1: (100)
mlp1 = MLPClassifier(hidden_layer_sizes=(100), max_iter=3000,)
mlp1.fit(X_train_scaled, y_train)
pred1 = mlp1.predict(X_test_scaled)
acc1 = accuracy_score(y_test, pred1)
print(f"hidden_layer_sizes (100): accuracy {acc1:.4f}\n")
```

hidden_layer_sizes (100): accuracy 0.6615

```
#Experiment with Structure 2: (50, 25)
mlp2 = MLPClassifier(hidden_layer_sizes=(50, 25), max_iter=3000, random_state=42)
mlp2.fit(X_train_scaled, y_train)
pred2 = mlp2.predict(X_test_scaled)
acc2 = accuracy_score(y_test, pred2)
print(f"hidden_layer_sizes (50, 25): accuracy {acc2:.4f}\n")
```

hidden_layer_sizes (50, 25): accuracy 0.7031

```
#Experiment with Structure 2: (30, 30, 30)
mlp3 = MLPClassifier(hidden_layer_sizes=(30, 30, 30), max_iter=3000, random_state=42)
mlp3.fit(X_train_scaled, y_train)
pred3 = mlp3.predict(X_test_scaled)
acc3 = accuracy_score(y_test, pred3)
print(f"hidden_layer_sizes (30, 30, 30): accuracy {acc3:.4f}\n")
```

hidden_layer_sizes (30, 30, 30): accuracy 0.6615

Premiga pengtab 673040672-0

```
print("===== summary =====")
print(f"Hidden layer Structure      Accuracy")
print(f"(100)                        {acc1:.4f}")
print(f"(50, 25)                      {acc2:.4f}")
print(f"(30, 30, 30)                  {acc3:.4f}")

results = {
    "(100,)": acc1,
    "(50, 25)": acc2,
    "(30, 30, 30)": acc3
}

best_design_name = max(results, key=results.get)
best_accuracy = results[best_design_name]

print("-----")
print(f"Optimal Hidden Layer Structure : {best_design_name}")
print(f"Highest Achieved Accuracy : {best_accuracy:.4f} ")
print("-----\n")
```

```
===== summary =====
Hidden layer Structure      Accuracy
(100)                      0.6719
(50, 25)                   0.7031
(30, 30, 30)              0.6615
-----
Optimal Hidden Layer Structure : (50, 25)
Highest Achieved Accuracy: 0.7031
-----
```